

Class Design in C++

Understanding Interfaces

When you're designing a class in C++, the first thing you should decide is the public interface for the class. The public interface determines how your class will be used by other programmers (or you), and once designed and implemented it should generally stay pretty constant. You may decide to add to the interface, but once you've started using the class, it will be hard to remove functions from the public interface (unless they aren't used and weren't necessary in the first place).

But that doesn't mean that you should include more functionality in your class than necessary just so that you can later decide what to remove from the interface. If you do this, you'll just make the class harder to use. People will ask questions like, "why are there four ways of doing this? Which one is better? How can I choose between them?" It's usually easier to keep things simple and provide one way of doing each thing unless there's a compelling reason why your class should offer multiple methods with the same basic functionality.

At the same time, just because adding methods to the public interface (probably) won't break anything that doesn't mean that you should start off with a tiny interface. First of all, if anybody decides to inherit from your class and you then choose a function with the same name, you're in for a boatload of confusion. First, if you don't declare the function virtual, then an object of the subclass will have the function chosen depending on the static type of the pointer. This can be messy. Moreover, if you do declare it virtual, then you have the issue that it might provide a different type of functionality than was intended by the original implementation of that function. Finally, you just can't add a pure virtual function to a class that's already in use because nobody who has inherited from it will have implemented that function.

The public interface, then, should remain as constant as possible. In fact, a good approach to designing classes is to write the interface before the implementation because it's what determines how your class interacts with the rest of the world (which is more important for the program as a whole than how the class is actually implemented). Moreover, if you write the interface first, you can get a feel for how the class will work with other classes before you actually dive into the implementation details.

Inheritance and Class Design

The second issue of your class design is what should be available to programmers who wish to create subclasses. This interface is primarily determined by virtual functions, but you can also include protected methods that are designed for use by the class or its subclasses (remember that protected methods are visible to subclasses while private methods are not).

A key consideration is whether it makes sense for a function to be virtual. A function should be virtual when the implementation is likely to differ from subclass to subclass. Vice-versa, whenever a function should not change, then it should be made non-virtual. The key idea is to think about whether to make a function virtual by asking if the function should always be the same for every class.

For example, if you have a class is designed to allow users to monitor network traffic and you want to allow subclasses that implement different ways of analyzing the traffic, you might use the following interface:

```
class TrafficWatch
{
    public:
        // Packet is some class that implements information about network
        // packets
        void addPacket (const Packet& network_packet);

        int getAveragePacketSize ();
}
```

```
int getMaxPacket ();  
  
virtual bool isOverloaded ();  
};
```

In this class, some methods will not change from implementation to implementation; adding a packet should always be handled the same way, and the average packet size isn't going to change either. On the other hand, someone might have a very different idea of what it means to have an overloaded network. This will change from situation to situation and we don't want to prevent someone from changing how this is computed--for some, anything over 10 Mbits/sec of traffic might be an overloaded network, and for others, it would require 100 Mbits/sec on some specific network cables.

Finally, when publicly inheriting from any class or designing for inheritance, remember that you should strive for it to be clear that inheritance models is-a. At heart, the is-a relationship means that the subclass should be able to appear anywhere the parent class could appear. From the standpoint of the user of the class, it should not matter whether a class is the parent class or a subclass.

To design an is-a relationship, make sure that it makes sense for the class to include certain functions to be sure that it doesn't include that subclasses might not actually need. One example of having an extra function is that of a Bird class that implements a fly function. The problem is that not all birds can fly--penguins and emus, for instance. This suggests that a more prudent design choice might be to have two subclasses of birds, one for birds that can fly and one for flightless birds. Of course, it might be overkill to have two subclasses of bird depending on how complex your class hierarchy will be. If you know that nobody would ever expect use your class for a flightless bird, then it's not so bad. Of course, you won't always know what someone will use your class for and it's much easier to think carefully before you start to implement an entire class hierarchy than it will be to go back and change it once people are using it.